Processes and Services Programming

Activity 3. Multiprocess Programming

1. Create a Python program that uses the `fork` function to generate a specified number of child processes from a parent process. Each child process must print its PID and the PID of its parent process. Requirements:

a. The program must ask the user how many child processes they want to create.
b. Each time a new child process is created, it must print a message to the console in the following format: Child: [child PID], Parent: [parent PID]
c. The parent process must wait for all child processes to finish before terminating (os.wait()).
d. Implement an option for the user to stop the creation of child processes before reaching the requested number. If the user chooses not to create any more processes, the program must exit the loop and wait for all processes to terminate.

2. Create a Python program that uses `fork` to create a child process that performs a simple task (for example, counting from 1 to 5). The parent process must wait for the child process to complete its task and then print a message indicating that the child process has finished. Requirements:

a. The child process must count from 1 to 5, printing each number with a short delay (time.sleep(1)).
b. The parent process must wait for the child process to finish before printing the final message: The child process has finished.

3. Create a Python program that uses `multiprocessing.Process` to generate a specified number of child processes from a parent process. Each child process must print its PID and the PID of its parent process. Requirements:

a. The programme must ask the user how many child processes they want to create.
b. Each child process must print a message to the console in the following format: Child: [Child PID], Parent: [Parent PID]
c. The parent process must wait for all child processes to finish before terminating (p.join()).
d. Implement an option for the user to stop the creation of child processes before reaching the requested number. If the user chooses not to create any more processes, the programme must exit the loop and wait for all processes to terminate.

4. Create a Python program that uses `multiprocessing.Process` to create a child process that counts from 1 to N and calculates que addition of the numbers counted. The parent process must wait for the child process to complete the task and then print the total sum. Requirements:

a. The parent process must ask the user up to which number (N) the child process should count.
b. The child process must count from 1 to N, adding the numbers along the way.
c. The parent process must wait for the child process to finish using p.join().
d. Upon completion, the parent process must print the total sum obtained by the child process.
e. The child process must print the total sum obtained by the child process.

5. Create a Python program that uses `subprocess.run` to execute various commands on the system. The programme should ask the user which command they want to execute and handle exceptions appropriately if the command is invalid or does not exist. Requirements:

a. The program must allow the user to enter a command and its parameters (for example, ping www.google.com -c 5).
b. Use subprocess.run to execute the command entered by the user.
c. Implement a try-except block to catch possible errors, such as the absence of the command or execution errors, and print an appropriate error message.
d. After executing the command, the program should ask the user if they wish to execute another command or exit the program.

6. Create a Python program that uses the `psutil` library to list all processes currently running on the system. For each process, the program should display the process name, its PID, and the current priority. Requirements:

a. Use psutil.process_iter() to iterate over the running processes.
b. For each process, obtain and display: The process name.the process PID. the process priority (use proc.nice()).
c. Handle any exceptions that may occur when accessing process information (e.g., processes that terminate before they can be accessed)..

7. Develop a Python program that uses the `psutil` library to change the priority of a specific process. The user must be able to enter the PID of the process and the new priority they wish to set. Requirements:

a. Ask the user for the PID of the process whose priority level they wish to change.
b. Ask the user for the new priority they wish to set (you can define valid ranges, such as values between -20 and 19).
c. Use psutil to access the process and change its priority with the nice() method.
d. Print the new priority of the process after making the change.
e. Handle any exceptions that may occur, such as an invalid PID or a permissions error when attempting to change the priority.

8. Create a Python program that lists all running processes and allows the user to select one to terminate. Use the `psutil` library to manage the processes. Requirements:

a. Use psutil.process_iter() to list all running processes along with their names and PIDs.
b. Ask the user for the name of the process they want to terminate.
c. Check if the process exists and, if so, kill it using the kill() method.
d. Handle exceptions appropriately (e.g., if the process does not exist or if you do not have permission to kill it).
e. Print a message confirming that the process has been killed or that an error has occurred.

9. Develop a Python program that allows the user to kill a process and then monitor its completion. The programme should display a message indicating whether the process has finished. Requirements:

a. Use psutil to list running processes and allow the user to select a process by its PID.
b. Ask the user if they want to kill the selected process.
c. If the user chooses to terminate the process, use the kill() method to terminate it.
d. After terminating the process, use the poll() method in a loop to check whether the process has finished. Print a message indicating that the process has finished.
e. Handle any exceptions that may occur, such as non-existent processes or lack of permissions.

10. Create a Python program that executes a system command both synchronously and asynchronously. Use `subprocess` and `asyncio` to manage the execution. Requirements:

a. Define a function execute_synchronous_command(command) that uses subprocess.run to execute a command on the system and display its output.
b. Define an asynchronous function execute_asynchronous_command(command) that uses asyncio.create_subprocess_exec to execute the same command asynchronously.
c. In the main() function, ask the user to enter a command they want to execute (for example, ping www.google.com -c 4 for Linux).
d. First, execute the command synchronously and display its output.
e. Then, execute the same command asynchronously and display its output.
f. Be sure to handle any exceptions that may occur during execution.

11. Develop a Python program that uses `asyncio` to open multiple applications (e.g., a text editor and a terminal) asynchronously. Requirements:

a. Define an asynchronous function open_application(application_name) that uses asyncio.create_subprocess_exec to open an application based on the name it receives as an argument.
b. In the main() function, ask the user how many applications they want to open and request the name of each application (e.g., notepad, gedit, cmd, etc.).
c. Call open_application for each application entered, allowing them to open simultaneously.
d. Print the PID of each process created.
e. When all applications have been opened, wait for the user to press a key to close the program.

12. Create a Python program that uses `pipes` to communicate between two processes. The first process must generate a list of random numbers and send them to the second process, which will calculate the sum of those numbers. Requirements:

a. Use subprocess.Popen to create two processes: The first process generates a list of random numbers and sends it through the pipe. The second process receives the list and calculates the sum of the numbers.
b. The first process must print the generated numbers before sending them.
c. The second process must print the total sum of the numbers received.
d. Be sure to handle possible exceptions and errors.

13. Develop a program that allows the user to enter text, and then launches a child process that counts the number of words in the text provided. The child process must send the count back to the parent process. Requirements:

a. The parent process must prompt the user to enter text.
b. Use subprocess.run to create the child process, passing the text as input.
c. The child process must count the words and return the result to the parent process.
d. Print the number of words in the text provided in the parent process.
e. Handle exceptions that may occur during execution.

14. Create a program that executes an external script (e.g., script.py) and captures its output. The script should print the numbers 1 to 5, and the main process should print this output to the console. Requirements:

a. Create a file called script.py that contains a loop that prints the numbers from 1 to 5.
b. Use subprocess.run in the main programme to execute script.py.
c. Capture the output of the script and display it on the console.
d. Ensure that you handle any exceptions that may occur when executing the script.